

CHƯƠNG 11: LẬP KẾ HOẠCH TỰ ĐỘNG

TRÍ TUỆ NHÂN TẠO - ARTIFICIAL INTELLIGENCE

Nội dung Chương 11

- ◇ 11.1 Định nghĩa về Lập kế hoạch Cổ điển (Classical Planning)
- ◇ 11.2 Các thuật toán Lập kế hoạch Cổ điển
- ◇ 11.3 Các hàm tự suy (Heuristics) cho Lập kế hoạch
- ◇ 11.4 Lập kế hoạch Phân cấp (Hierarchical Planning)
- ◇ 11.5 Lập kế hoạch trong Môi trường Không tất định
- ◇ 11.6 Thời gian, Lịch trình và Tài nguyên

11.1 Lập kế hoạch Cổ điển (Classical Planning)

◇ Lập kế hoạch là quá trình tìm kiếm một chuỗi hành động để đạt được một mục tiêu cụ thể.

◇ **Ngôn ngữ PDDL (Planning Domain Definition Language):** Ngôn ngữ tiêu chuẩn để biểu diễn các bài toán lập kế hoạch.

◇ **Cấu trúc bài toán:**

- **Trạng thái (States):** Tập hợp các biến logic mô tả thế giới.
- **Hành động (Actions):** Bao gồm *Tiền điều kiện (Preconditions)* và *Hiệu ứng (Effects)*.

◇ **Ví dụ kinh điển:** Bài toán Vận chuyển hàng không (Air cargo), Lốp dự phòng (Spare tire) và Thế giới khối vuông (Blocks world).

Ví dụ PDDL: Vận chuyển Hàng không

Init(*At*(*C*₁, *SFO*) $\wedge$ *At*(*C*₂, *JFK*) $\wedge$ *At*(*P*₁, *SFO*) $\wedge$ *At*(*P*₂, *JFK*)
 $\wedge$ *Cargo*(*C*₁) $\wedge$ *Cargo*(*C*₂) $\wedge$ *Plane*(*P*₁) $\wedge$ *Plane*(*P*₂)
 $\wedge$ *Airport*(*JFK*) $\wedge$ *Airport*(*SFO*))
Goal(*At*(*C*₁, *JFK*) $\wedge$ *At*(*C*₂, *SFO*))
Action(*Load*(*c*, *p*, *a*),
 PRECOND: *At*(*c*, *a*) $\wedge$ *At*(*p*, *a*) $\wedge$ *Cargo*(*c*) $\wedge$ *Plane*(*p*) $\wedge$ *Airport*(*a*)
 EFFECT: $\neg$ *At*(*c*, *a*) $\wedge$ *In*(*c*, *p*))
Action(*Unload*(*c*, *p*, *a*),
 PRECOND: *In*(*c*, *p*) $\wedge$ *At*(*p*, *a*) $\wedge$ *Cargo*(*c*) $\wedge$ *Plane*(*p*) $\wedge$ *Airport*(*a*)
 EFFECT: *At*(*c*, *a*) $\wedge$ $\neg$ *In*(*c*, *p*))
Action(*Fly*(*p*, *from*, *to*),
 PRECOND: *At*(*p*, *from*) $\wedge$ *Plane*(*p*) $\wedge$ *Airport*(*from*) $\wedge$ *Airport*(*to*)
 EFFECT: $\neg$ *At*(*p*, *from*) $\wedge$ *At*(*p*, *to*))

Figure 1: Mô tả PDDL của một bài toán lập kế hoạch vận chuyển hàng không (Air cargo).

Ví dụ PDDL: Lốp Dự phòng

Init(*Tire*(*Flat*) $\wedge$ *Tire*(*Spare*) $\wedge$ *At*(*Flat*,*Axle*) $\wedge$ *At*(*Spare*,*Trunk*))
Goal(*At*(*Spare*,*Axle*))
Action(*Remove*(*obj*,*loc*),
 PRECOND: *At*(*obj*,*loc*)
 EFFECT: $\neg$ *At*(*obj*,*loc*) $\wedge$ *At*(*obj*,*Ground*))
Action(*PutOn*(*t*, *Axle*),
 PRECOND: *Tire*(*t*) $\wedge$ *At*(*t*,*Ground*) $\wedge$ $\neg$ *At*(*Flat*,*Axle*) $\wedge$ $\neg$ *At*(*Spare*,*Axle*)
 EFFECT: $\neg$ *At*(*t*,*Ground*) $\wedge$ *At*(*t*,*Axle*))
Action(*LeaveOvernight*,
 PRECOND:
 EFFECT: $\neg$ *At*(*Spare*,*Ground*) $\wedge$ $\neg$ *At*(*Spare*,*Axle*) $\wedge$ $\neg$ *At*(*Spare*,*Trunk*)
 $\wedge$ $\neg$ *At*(*Flat*,*Ground*) $\wedge$ $\neg$ *At*(*Flat*,*Axle*) $\wedge$ $\neg$ *At*(*Flat*, *Trunk*))

Figure 2: Bài toán lốp dự phòng đơn giản (Spare tire problem).

Thế giới Khối vuông (Blocks World)

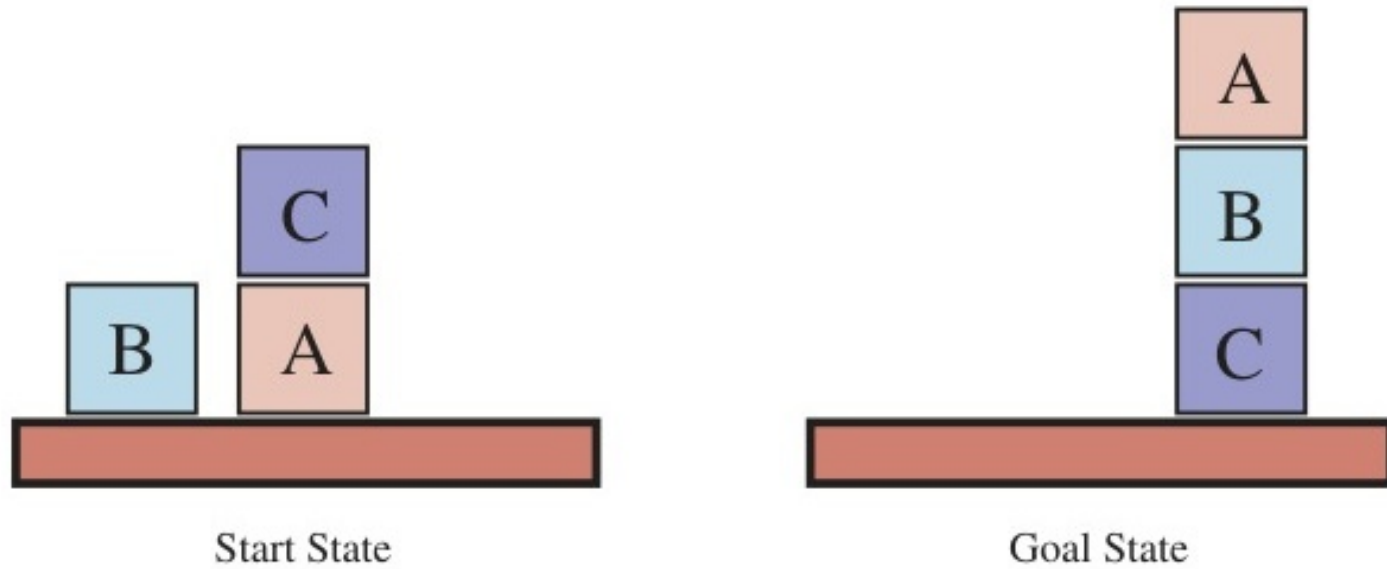


Figure 3: Sơ đồ của bài toán thế giới khối vuông.

Kế hoạch Xây Tháp Khối Vuông

Init(*On*(*A*, *Table*) $\wedge$ *On*(*B*, *Table*) $\wedge$ *On*(*C*, *A*)
 $\wedge$ *Block*(*A*) $\wedge$ *Block*(*B*) $\wedge$ *Block*(*C*) $\wedge$ *Clear*(*B*) $\wedge$ *Clear*(*C*) $\wedge$ *Clear*(*Table*))
Goal(*On*(*A*, *B*) $\wedge$ *On*(*B*, *C*))
Action(*Move*(*b*, *x*, *y*),
 PRECOND: *On*(*b*, *x*) $\wedge$ *Clear*(*b*) $\wedge$ *Clear*(*y*) $\wedge$ *Block*(*b*) $\wedge$ *Block*(*y*) $\wedge$
 (*b* $\neq$ *x*) $\wedge$ (*b* $\neq$ *y*) $\wedge$ (*x* $\neq$ *y*),
 EFFECT: *On*(*b*, *y*) $\wedge$ *Clear*(*x*) $\wedge$ $\neg$ *On*(*b*, *x*) $\wedge$ $\neg$ *Clear*(*y*))
Action(*MoveToTable*(*b*, *x*),
 PRECOND: *On*(*b*, *x*) $\wedge$ *Clear*(*b*) $\wedge$ *Block*(*b*) $\wedge$ *Block*(*x*),
 EFFECT: *On*(*b*, *Table*) $\wedge$ *Clear*(*x*) $\wedge$ $\neg$ *On*(*b*, *x*))

Figure 4: Một bài toán lập kế hoạch trong thế giới khối vuông: xây dựng một tháp ba khối.

11.2 Các thuật toán Lập kế hoạch Cổ điển

◇ Tìm kiếm không gian trạng thái Tiến (Forward state-space search):

- Bắt đầu từ trạng thái ban đầu, áp dụng các hành động hợp lệ cho đến khi đạt được mục tiêu.
- Thường bị bùng nổ tổ hợp nếu không có heuristic tốt.

◇ Tìm kiếm Lùi (Backward search):

- Bắt đầu từ mục tiêu, tìm các hành động có hiệu ứng đáp ứng mục tiêu, và tiếp tục lùi về trạng thái ban đầu.
- Chỉ xem xét các hành động có liên quan (Relevant actions).

◇ **Biểu diễn khác:** Lập kế hoạch dưới dạng Bài toán Thỏa mãn Boolean (SAT) hoặc Mạng kế hoạch (Planning Graph).

Tìm kiếm Tiến và Lùi trong Lập kế hoạch

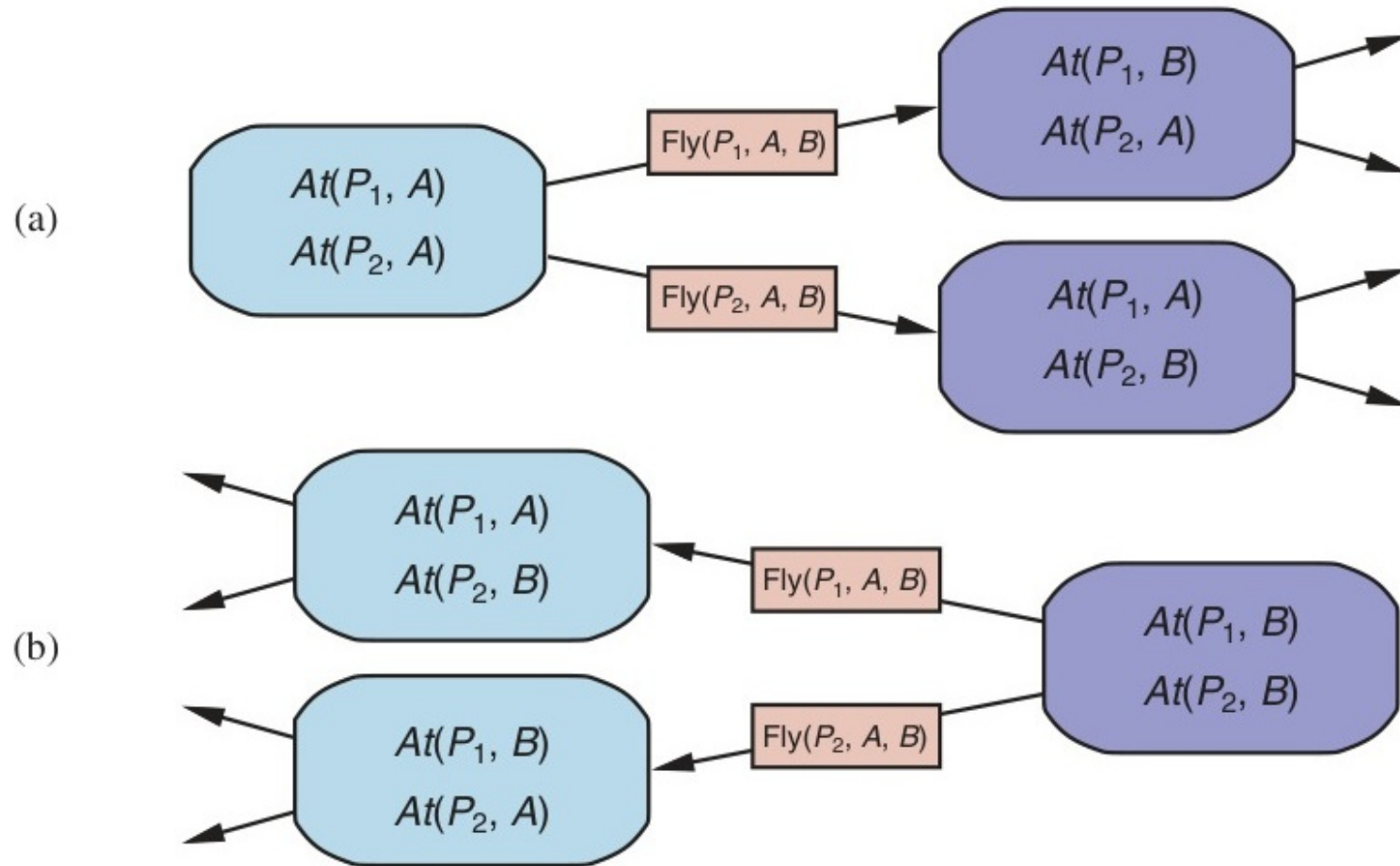


Figure 5: Hai phương pháp tìm kiếm kế hoạch: Tiến (Forward) từ trạng thái ban đầu và Lùi (Backward) từ mục tiêu.

11.3 Các hàm tự suy (Heuristics) cho Lập kế hoạch

◇ Việc tính toán hàm heuristic $h(s)$ tự động là điểm mấu chốt của lập kế hoạch hiện đại.

◇ **Khái niệm Nới lỏng (Relaxation):**

- Giải một bài toán dễ hơn để lấy chi phí làm heuristic cho bài toán gốc.
- *Ví dụ:* Bỏ qua tất cả các *Tiền điều kiện* (Preconditions) hoặc bỏ qua các *Hiệu ứng phủ định* (Negative effects).

◇ **Cắt tỉa nhánh độc lập với miền (Domain-independent pruning):** Loại bỏ các hành động thừa hoặc không thể dẫn đến mục tiêu mà không cần biết chi tiết về miền ứng dụng.

Heuristic: Bỏ qua Danh sách Xóa

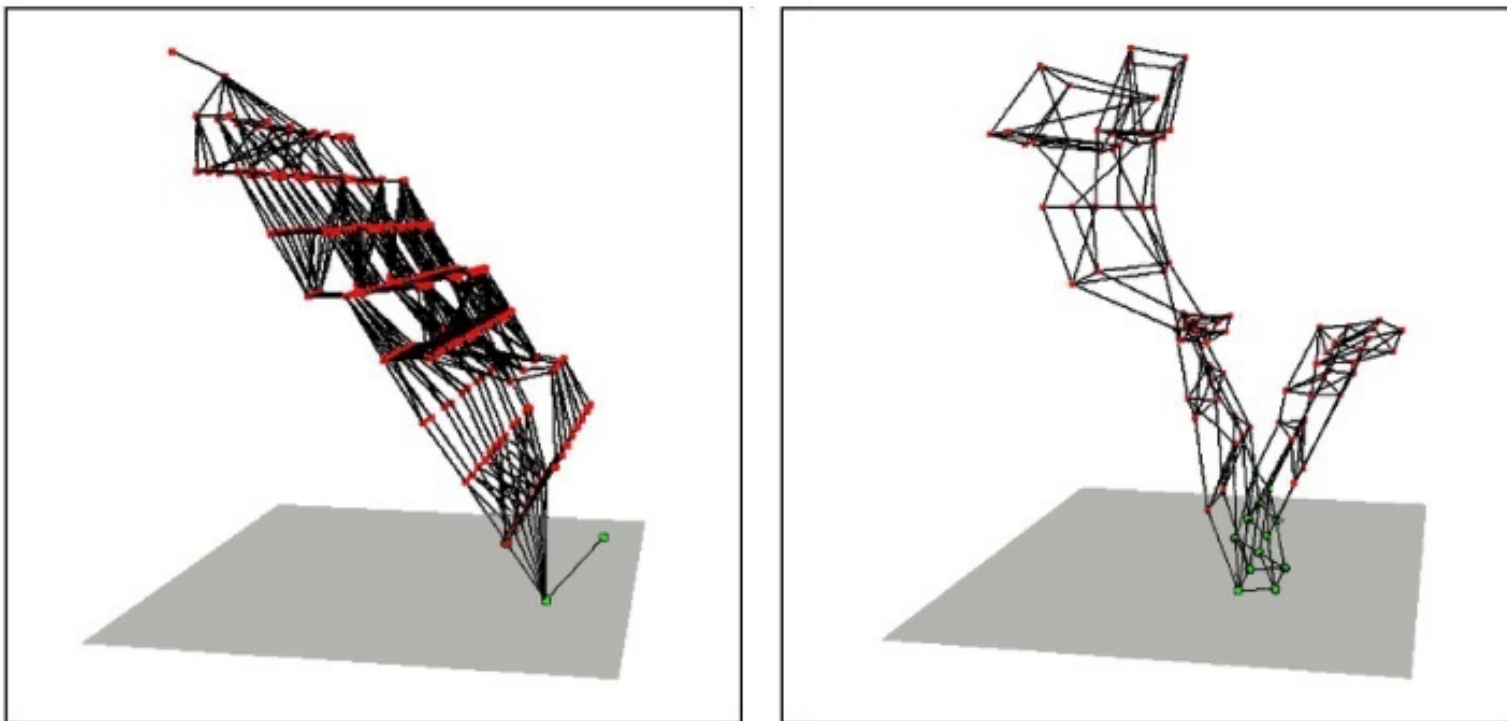


Figure 6: Hai không gian trạng thái từ các bài toán lập kế hoạch sử dụng heuristic bỏ qua danh sách xóa (ignore-delete-lists).

11.4 Lập kế hoạch Phân cấp (Hierarchical Planning)

◇ Trong thực tế, con người không lập kế hoạch từng bước nhỏ lẻ, mà lên kế hoạch theo các cấp độ.

◇ **Hành động Cấp cao (High-level actions - HLA):**

- Chia nhỏ một nhiệm vụ phức tạp (ví dụ: "Xây nhà") thành các HLA ("Đổ móng", "Xây tường", "Lợp mái").

◇ **Quá trình Tinh chỉnh (Refinement):**

- Phân tích từng HLA thành các Hành động nguyên thủy (Primitive actions) để có thể thực thi được.
- Hỗ trợ tạo ra Thư viện kế hoạch (Plan library) có thể tái sử dụng.

Tinh chỉnh Hành động Cấp cao

$Refinement(Go(Home, SFO),$
 STEPS: $[Drive(Home, SFO LongTermParking),$
 $Shuttle(SFO LongTermParking, SFO)]$)
 $Refinement(Go(Home, SFO),$
 STEPS: $[Taxi(Home, SFO)]$)

 $Refinement(Navigate([a, b], [x, y]),$
 PRECOND: $a = x \wedge b = y$
 STEPS: $[]$)
 $Refinement(Navigate([a, b], [x, y]),$
 PRECOND: $Connected([a, b], [a - 1, b])$
 STEPS: $[Left, Navigate([a - 1, b], [x, y])]$)
 $Refinement(Navigate([a, b], [x, y]),$
 PRECOND: $Connected([a, b], [a + 1, b])$
 STEPS: $[Right, Navigate([a + 1, b], [x, y])]$)
 ...

Figure 7: Định nghĩa các tinh chỉnh (refinements) khả thi cho hai hành động cấp cao.

Thuật toán Tìm kiếm Lập kế hoạch Phân cấp

function HIERARCHICAL-SEARCH(*problem, hierarchy*) **returns** a solution or *failure*

frontier $\leftarrow$ a FIFO queue with [*Act*] as the only element

while true do

if IS-EMPTY(*frontier*) **then return failure**

plan $\leftarrow$ POP(*frontier*) // chooses the shallowest plan in frontier

hla $\leftarrow$ the first HLA in *plan*, or *null* if none

prefix, suffix $\leftarrow$ the action subsequences before and after *hla* in *plan*

outcome $\leftarrow$ RESULT(*problem*.INITIAL, *prefix*)

if *hla* is *null* **then** // so plan is primitive and outcome is its result

if *problem*.IS-GOAL(*outcome*) **then return plan**

else for each sequence in REFINEMENTS(*hla, outcome, hierarchy*) **do**

 add APPEND(*prefix, sequence, suffix*) to *frontier*

Figure 8: Một cài đặt theo chiều rộng (breadth-first) của thuật toán tìm kiếm lập kế hoạch phân cấp thuận.

Tập hợp Có thể Đạt được (Reachable Sets)

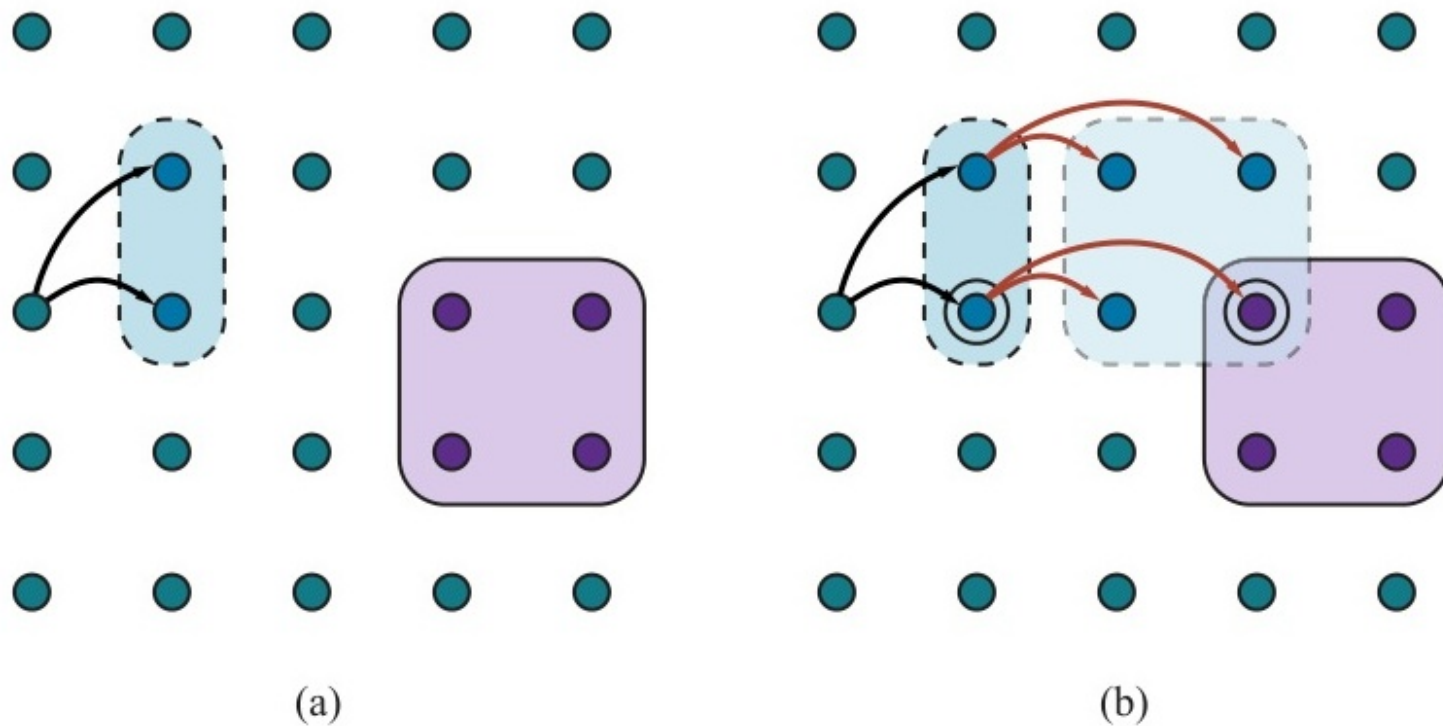


Figure 9: Ví dụ sơ đồ về các tập hợp có thể đạt được (reachable sets) trong quá trình tìm kiếm.

Đạt Mục tiêu với Mô tả Xấp xỉ

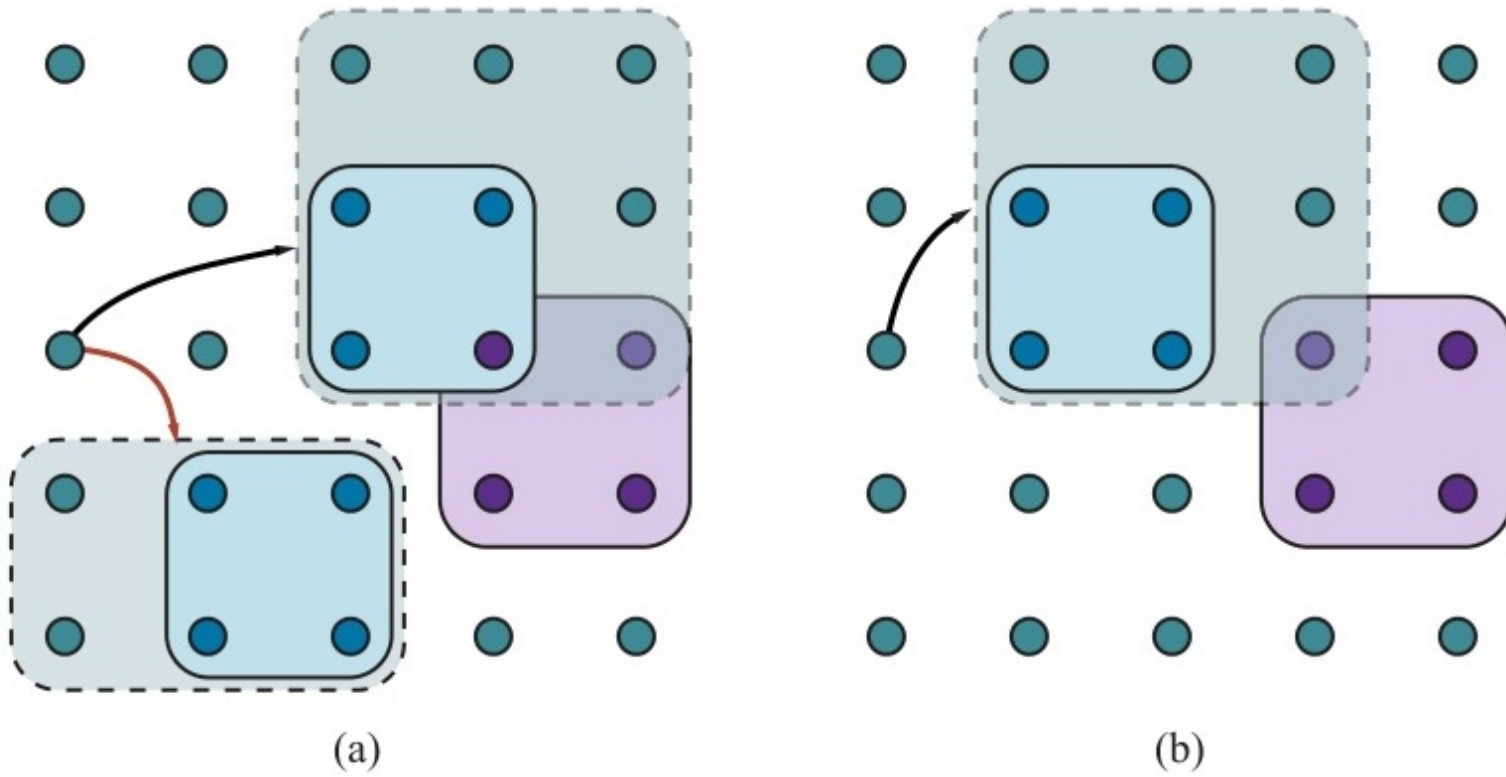


Figure 10: Đạt được mục tiêu đối với các kế hoạch cấp cao sử dụng mô tả xấp xỉ (approximate descriptions).

11.5 Lập kế hoạch trong Môi trường Không tất định

- ◇ Khi môi trường không hoàn toàn quan sát được hoặc hành động có thể thất bại.
- ◇ **Lập kế hoạch phi cảm biến (Sensorless / Conformant planning):** Tìm một chuỗi hành động luôn thành công dù trạng thái ban đầu là gì (không cần cảm biến).
- ◇ **Lập kế hoạch dự phòng (Contingent planning):** Xây dựng kế hoạch theo cấu trúc rẽ nhánh dựa trên kết quả nhận thức (*Nếu thấy A thì làm X, ngược lại làm Y*).
- ◇ **Lập kế hoạch trực tuyến (Online planning) và Giám sát:** Vừa thực thi vừa giám sát (Execution monitoring). Nếu phát hiện lỗi (cảm biến báo về không như dự kiến), hệ thống sẽ *Lập kế hoạch lại (Re-planning)*.

Lập kế hoạch Phân cấp với Ngữ nghĩa Thiên thần

```

function ANGELIC-SEARCH(problem, hierarchy, initialPlan) returns a solution or fail
frontier  $\leftarrow$  a FIFO queue with initialPlan as the only element
while true do
  if IS-EMPTY?(frontier) then return fail
  plan  $\leftarrow$  POP(frontier)      // chooses the shallowest node in frontier
  if REACH+(problem.INITIAL, plan) intersects problem.GOAL then
    if plan is primitive then return plan      // REACH+ is exact for primitive plans
    guaranteed  $\leftarrow$  REACH-(problem.INITIAL, plan)  $\cap$  problem.GOAL
    if guaranteed  $\neq \{\}$  and MAKING-PROGRESS(plan, initialPlan) then
      finalState  $\leftarrow$  any element of guaranteed
      return DECOMPOSE(hierarchy, problem.INITIAL, plan, finalState)
    hla  $\leftarrow$  some HLA in plan
    prefix, suffix  $\leftarrow$  the action subsequences before and after hla in plan
    outcome  $\leftarrow$  RESULT(problem.INITIAL, prefix)
    for each sequence in REFINEMENTS(hla, outcome, hierarchy) do
      add APPEND(prefix, sequence, suffix) to frontier

function DECOMPOSE(hierarchy, s0, plan, sf) returns a solution
solution  $\leftarrow$  an empty plan
while plan is not empty do
  action  $\leftarrow$  REMOVE-LAST(plan)
  si  $\leftarrow$  a state in REACH-(s0, plan) such that sf  $\in$  REACH-(si, action)
  problem  $\leftarrow$  a problem with INITIAL = si and GOAL = sf
  solution  $\leftarrow$  APPEND(ANGELIC-SEARCH(problem, hierarchy, action), solution)
  sf  $\leftarrow$  si
return solution

```

Figure 11: Thuật toán lập kế hoạch phân cấp sử dụng ngữ nghĩa "thiên thần" (angelic semantics) để xác định các kế hoạch cấp cao hiệu quả.

Giám sát Hành động (Action Monitoring)

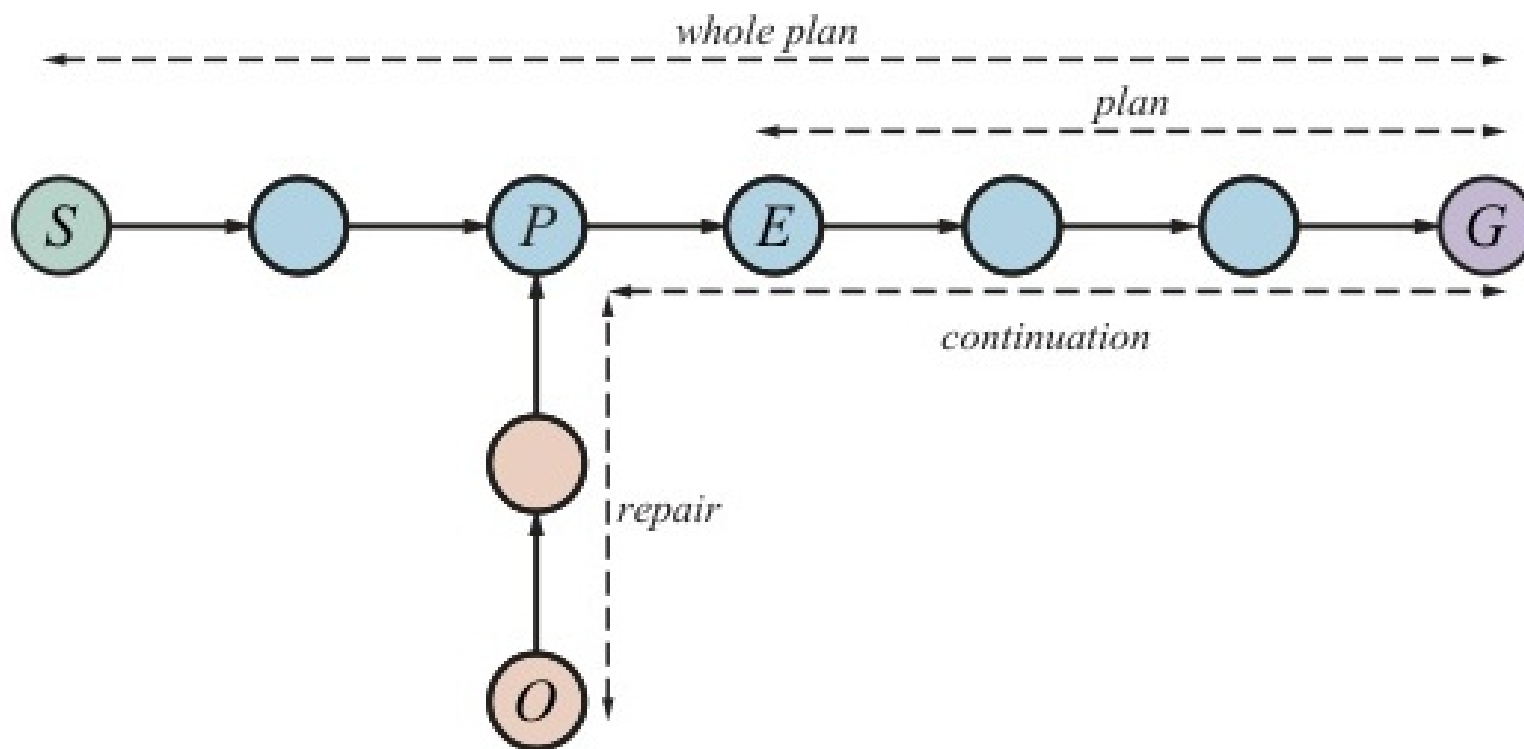


Figure 12: Sơ đồ của quá trình giám sát hành động: Khi kế hoạch gặp sự cố, tác nhân cần lập kế hoạch lại.

11.6 Thời gian, Lịch trình và Tài nguyên

- ◇ Các bài toán PDDL cơ bản thường bỏ qua thời gian và tài nguyên.
- ◇ **Ràng buộc thời gian (Temporal planning):**
 - Các hành động có thể diễn ra song song và cần thời lượng cụ thể.
 - **Phương pháp Đường gang (Critical Path Method - CPM):** Tính Thời điểm bắt đầu sớm nhất (ES), Bắt đầu muộn nhất (LS) và Thời gian trễ (Slack).
- ◇ **Ràng buộc tài nguyên (Resource constraints):** Lập lịch biểu (Job-shop scheduling) chia sẻ các tài nguyên hữu hạn (Máy móc, Nhân công, Không gian).

Bài toán Lập Lịch biểu Công việc

Jobs($\{AddEngine1 \prec AddWheels1 \prec Inspect1\},$
 $\{AddEngine2 \prec AddWheels2 \prec Inspect2\}$)

Resources(*EngineHoists*(1), *WheelStations*(1), *Inspectors*(2), *LugNuts*(500))

Action(*AddEngine1*, DURATION:30,
USE:*EngineHoists*(1))

Action(*AddEngine2*, DURATION:60,
USE:*EngineHoists*(1))

Action(*AddWheels1*, DURATION:30,
CONSUME:*LugNuts*(20), USE:*WheelStations*(1))

Action(*AddWheels2*, DURATION:15,
CONSUME:*LugNuts*(20), USE:*WheelStations*(1))

Action(*Inspect_i*, DURATION:10,
USE:*Inspectors*(1))

Figure 13: Một bài toán lập lịch biểu (job-shop scheduling) để lắp ráp hai ô tô với các ràng buộc về tài nguyên.

Ràng buộc Thời gian & Đường Gang (Critical Path)

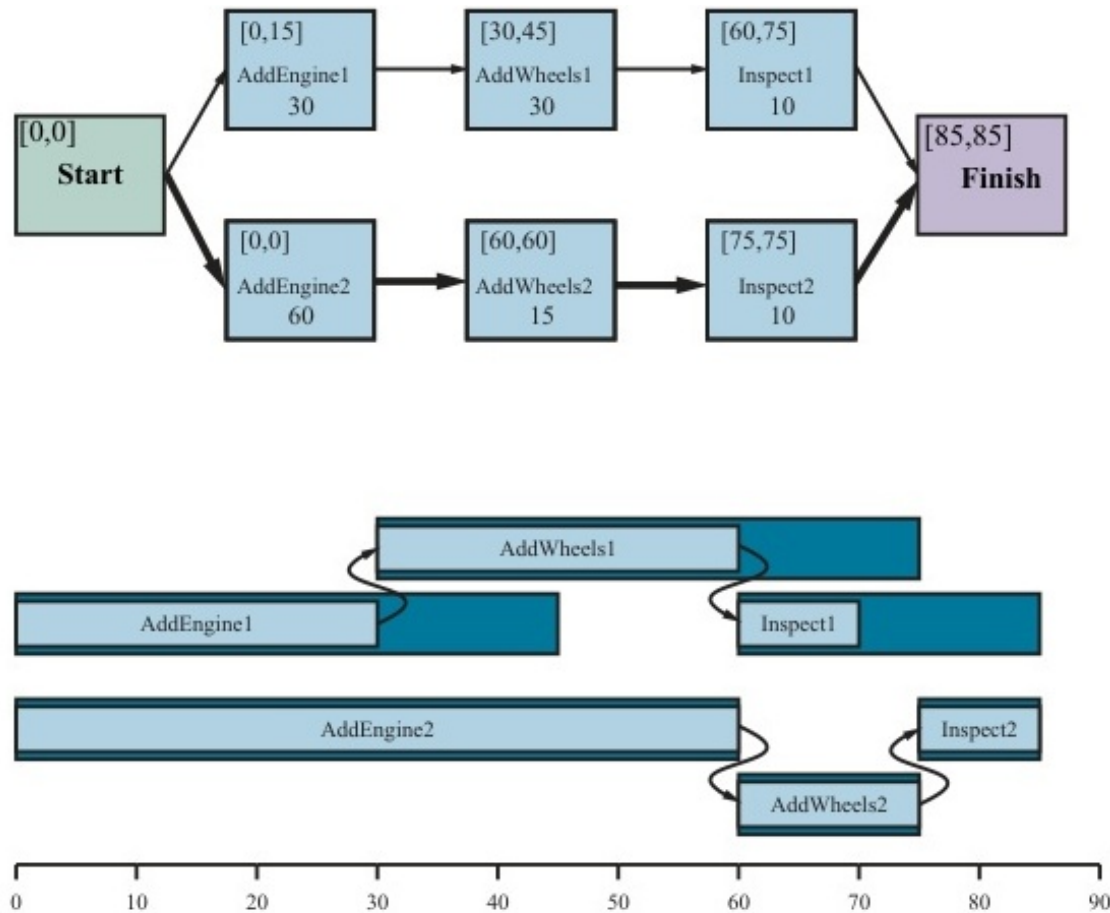


Figure 14: Biểu diễn các ràng buộc thời gian (CPM) cho bài toán lập lịch biểu. Các hành động trên đường găng không có thời gian trễ.

Giải pháp Lập Lịch Tối ưu

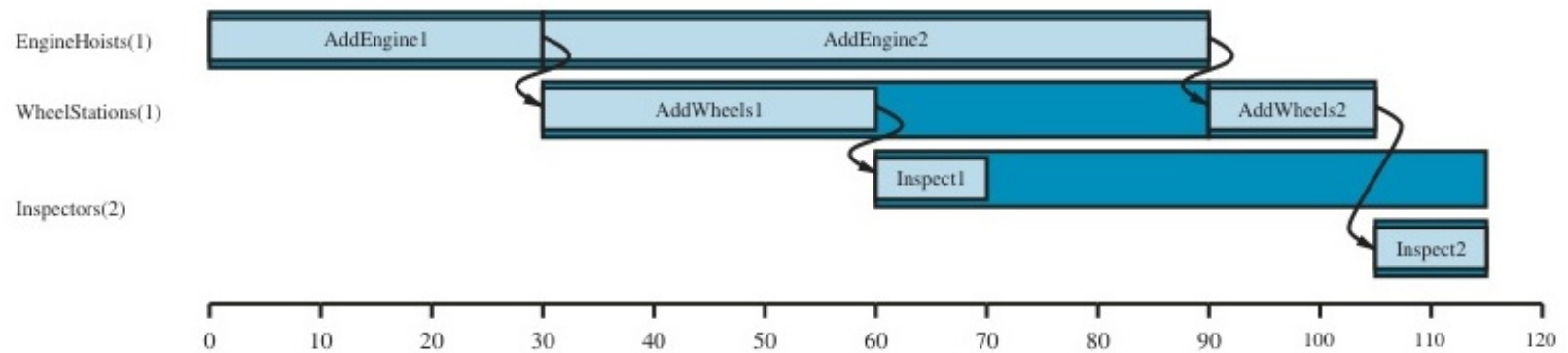


Figure 15: Một giải pháp cho bài toán lập lịch biểu có thời gian hoàn thành nhanh nhất (115 phút).

Tóm tắt Chương 11

◇ Hệ thống Lập kế hoạch tự động (Automated Planning) là minh chứng cho việc AI có thể tư duy dài hạn.

◇ **Điểm nhấn:**

- Chuyển từ Tìm kiếm thô sơ sang sử dụng Ngôn ngữ cấu trúc (PDDL).
- Khả năng tự động sinh hàm Heuristic thông qua nối lỏng bài toán.
- Khả năng rẽ nhánh, đối phó với sự cố ngoài ý muốn (Contingent planning).
- Khả năng lên lịch trình tối ưu với hàng vạn ràng buộc tài nguyên và thời gian (Critical Path Method).